Med Assist: An AI-Enhanced Consultation Tool for Online Doctor–Patient Meetings

BSc Computer Science Final Year Project

School of Computer Science, University of Birmingham

By: Essam Sheikh (2595328)

Supervisor: Mohammed Bahja

Abstract

Med Assist is a web application that aims to support doctors during and after online consultations using AI, without sending any patient data to third parties. It contains a locally hosted meeting bot which joins online google meetings and transcribes them with the choice of generating live or manual AI insights to aid clinical reasoning. The app provides weekly analytics and an AI chat interface to query a patient's consultations in natural language. These features aim to reduce documentation burden and highlight important clinical details which otherwise might have been missed.

Contents

1 Introduction and Background **2**
1.1 Core Components . . . 2
1.2 Motivation . . . 2

2 Problem Definition and Context **2**
2.1 Problems Addressed . . . 2
2.2 How the Technical Approach Evolved . . . 3
2.3 Related Work . . . 3
2.3.1 Meeting Bots and Transcription . . . 3
2.3.2 LLMs and RAG in Medicine . . . 3
2.3.3 Clinical Decision Support and NICE Guidelines . . . 3
2.4 Scope . . . 3

3 Design Goals and Requirements **4**
3.1 Functional Requirements . . . 4
3.2 Non-Functional Requirements . . . 4

4 System Architecture and Technologies **4**
4.1 Database Schema . . . 4
4.2 Overall Architecture . . . 5
4.3 Technology Stack . . . 5

5 Implementation of Core Features **6**
5.1 Meeting Bot, Audio Capture, and Transcription . . . 6
5.1.1 Phase 1: Chrome Extension . . . 6
5.1.2 Phase 2: Recall.ai . . . 6
5.1.3 Phase 3: Self-Hosted Vexa.ai . . . 6
5.2 AI Insights: Manual and Live Modes . . . 6
5.2.1 Manual AI Insights . . . 7
5.2.2 Live AI Insights . . . 7
5.2.3 Chrome Extension . . . 8
5.3 AI Analytics . . . 8
5.4 Ask AI . . . 8
5.4.1 Architecture . . . 8
5.4.2 Model Selection . . . 9
5.4.3 Engineering Optimisations . . . 9

6 Evaluation **9**
6.1 Manual Insights: Transcript Comparison . . . 9
6.2 Live Insights: GPT-4o Critic Agent . . . 10
6.3 Weekly Analytics: Comparative Check . . . 11
6.4 Ask AI: RAGAS . . . 11
6.5 Reflection . . . 11

7 Limitations and Future Work **12**
7.1 Current Limitations . . . 12
7.2 Future Work . . . 12

8 Conclusion **12**

References **13**

1 Introduction and Background

Remote GP consultations became common during COVID-19 and have stayed that way [1]. That is genuinely convenient, but it creates a documentation problem: the doctor is simultaneously listening to the patient, reasoning clinically, and knowing they will need to write everything up afterwards. Research on divided attention in clinical settings shows that this split focus leads to forgotten details and missed facts [2]. Med Assist was built to take the note-taking burden off the doctor while keeping all patient data local.

1.1 Core Components

The application has five main features:

1. **Meeting Bot and Transcription.** A self-hosted Vexa.ai bot joins the Google Meet call through a headless Playwright-driven Chromium browser, captures all audio, and transcribes it locally using Faster-Whisper. The result is a speaker-labelled transcript stored in SQLite.
2. **Manual AI Insights.** After the call, an LLM reads the transcript, the patient's background record, and relevant NICE guidelines fetched via FAISS using RAG, and produces a structured JSON problem list for the doctor to view.
3. **Live AI Insights.** During the call, a background worker runs every 12 seconds, processes only the newest transcript lines, and pushes a JSON update to a Chrome extension panel giving the doctor in-meeting prompts about emerging problems, risks, and suggested follow-up questions. These insights can also be viewed on the app.
4. **AI Analytics.** At the end of each week the system reads the `key_points` already generated by Manual Insights for each consultation and summarises the caseload — without ever touching the raw transcripts again.
5. **Ask AI Chatbot.** A LangGraph ReAct agent lets the doctor ask free-text questions about any patient. It searches for that patient's raw transcripts through FAISS and answers only from what is actually there.

1.2 Motivation

Patient Health Information (PHI) is covered by UK-GDPR and NHS Information Governance requirements. Sending consultation audio or transcripts to a third party carries a compliance and security risk that limits the adoption of many AI tools in primary care. Every component of Med Assist runs on the clinician's own infrastructure: the Vexa bot and Faster-Whisper in Docker, LLM inference through a local Ollama model, and all retrieval through local FAISS indices. No PHI touches an external server at any point. This ensures full control over sensitive data.

2 Problem Definition and Context

2.1 Problems Addressed

Three problems drove the project:

1. **Reliable audio capture without PHI leakage.** Browser-based meeting audio capture is difficult due to needing specific permissions and API access not generally available without using commercial products that conflict with NHS guidelines.
2. **Providing real-time AI support under confidentiality constraints.** Commercial transcription products like Nuance DAX and Microsoft Dragon send audio to central servers. That is not acceptable in many GP settings as they don't control the infrastructure.
3. **Querying patient histories quickly.** Doctors reviewing a returning patient currently sift through past notes by hand. Semantic search over embedded transcripts is faster.

2.2 How the Technical Approach Evolved

Three successive architectures were tried to capture and transcribe audio. By phase 2, the requirements changed to hosting everything locally after meeting with my supervisor.

Phase 1 — Chrome Extension. I developed a chrome extension to capture and then transcribe audio by using whisper API through my back-end. Capturing audio was tricky as Meet API isn't publicly available and chrome has restrictions on how an extension can capture audio. First I captured 1 general audio stream and put it through a speaker diarization model (assembly.ai [14]). The transcription worked but the labelling came back as a generic speaker 1/2. Instead I captured 2 audio streams and put them through whisper separately to ensure I know who is the doctor and who is the patient. The problem with this was combining both transcripts back together wasn't very accurate due to synchronisation issues.

Phase 2 — Recall.ai. [13] I found out about the recall API service that allows automatic video transcription by having an agent connect with the google meeting. The agent would extract the speaker ID and labelling accurately and after implementation it worked really well. After a few meetings, the approach was rejected as in a clinical setting, giving patient information to a third party is not allowed.

Phase 3 — Self-Hosted Vexa.ai. [3] is an open-source project that runs a Google Meet bot and a WhisperLive transcription stack entirely in Docker Compose on local hardware [4]. It uses the `small` Whisper model in CPU profile mode. Nothing leaves the machine.

2.3 Related Work

2.3.1 Meeting Bots and Transcription

Otter.ai, Fireflies.ai, and Recall.ai are the main commercial options for meeting transcription. Their data residency guarantees rarely satisfy NHS Information Governance requirements [5]. Academic interest in Ambient Clinical Intelligence — AI that passively documents consultations — has grown since transformer-based speech recognition became viable [6]. Nuance DAX Copilot and Microsoft Dragon are the most mature products in that space, but both are cloud-dependent and priced for NHS trust procurement, not individual practices.

2.3.2 LLMs and RAG in Medicine

Retrieval-Augmented Generation [7] is the standard approach for giving a language model access to specific facts at inference time without fine-tuning on sensitive data. Fine-tuning on PHI risks memorisation and leakage, so RAG is the only sensible option for a clinical tool. Med Assist uses it in two places: FAISS retrieval of NICE guideline chunks for Manual Insights, and FAISS retrieval over consultation transcripts for Ask AI.

2.3.3 Clinical Decision Support and NICE Guidelines

Clinical Decision Support Systems have a long research history [8]. The live challenge with NICE guidelines [9] is provenance: the doctor needs to know whether a recommendation comes from the transcript or from a guideline. Med Assist handles this through prompt bifurcation — the model is told explicitly to label the two sources differently. NICE retrieval doesn't happen for the live pipeline entirely, where latency makes the distinction harder to enforce clearly.

2.4 Scope

Med Assist is a decision-support tool only. It does not diagnose, prescribe, or recommend treatments. All output is advisory. The system covers Google Meet, English-language consultations, and single-doctor deployments on consumer hardware.

3 Design Goals and Requirements

3.1 Functional Requirements

1. A meeting bot that joins Google Meet autonomously, captures audio from all participants, and stores a speaker-labelled transcript in the local database.
2. Post-consultation Manual AI Insights that incorporate patient EHR data and NICE guidelines via FAISS retrieval, output as structured JSON.
3. Live AI Insights delivered during the call through a Chrome extension panel, generated by a non-blocking background task.
4. A weekly AI Analytics summary built from the Manual Insights already produced that week, not from raw transcripts.
5. An Ask AI agent that answers natural-language questions about a patient using only that patient's own transcripts as source material.

3.2 Non-Functional Requirements

Confidentiality. All PHI stays on the clinician's machine. The Vexa bot, Faster-Whisper, Ollama inference, and FAISS retrieval all run locally. The only external connection is to Google Meet itself (HTTPS). The bot is set to `voice_agent_enabled: false`, so it only listens — it never speaks into the call.

Clinical Safety. The system must not hallucinate diagnoses. Every LLM prompt carries an anti-inference rule and separates transcript-derived facts from guideline-derived suggestions.

Concurrency. SQLite runs in Write-Ahead Logging (WAL) mode so the LLM background workers and the Vexa transcript sync can write to the database at the same time without locking conflicts. This was a concrete problem that came up during development.

Latency. The 12-second throttle on live inference was set empirically — it is the shortest interval that reliably fits within the 3B model's CPU inference time. The Chrome extension polls every 5 seconds to keep the display fresh.

Hardware. Everything must run on a standard laptop: Intel Core Ultra 7, 16 GB RAM, no GPU. Running Vexa's WhisperLive, FastAPI with background workers, and Ollama simultaneously on one machine constrains which models are usable. Llama 3.2 (3B) is the largest model that returns a live insight in under 15 seconds on this hardware.

4 System Architecture and Technologies

4.1 Database Schema

The application uses SQLite via SQLAlchemy in WAL mode. The schema has three layers.

Core structural layer. `User` stores doctor accounts with a `specialty` field that feeds into AI prompts. `Patient` stores the `key_history` string which provides structured pre-existing diagnoses and medications that is prepended to every LLM prompt. `Consultation` links a patient to a doctor, tracks status (`draft`/`completed`), and holds the Vexa bot identifier.

Speech and insight layer. `TranscriptEntry` stores timestamped audio chunks with speaker labels. `AIInsight` is a one-to-one record per consultation holding the post-call summary, key points, and risk flags. `LiveInsight` is one-to-many: each background worker run that produces output adds a new row.

LLM interaction layer. `ChatSession` and `ChatMessage` back the Ask AI chat interface. `AIAnalytics` stores the weekly report with JSON blob columns for consultation and patient lists, avoiding schema changes as the caseload grows.

4.2 Overall Architecture

Figure 1 shows how the five layers connect. Everything talks over localhost.

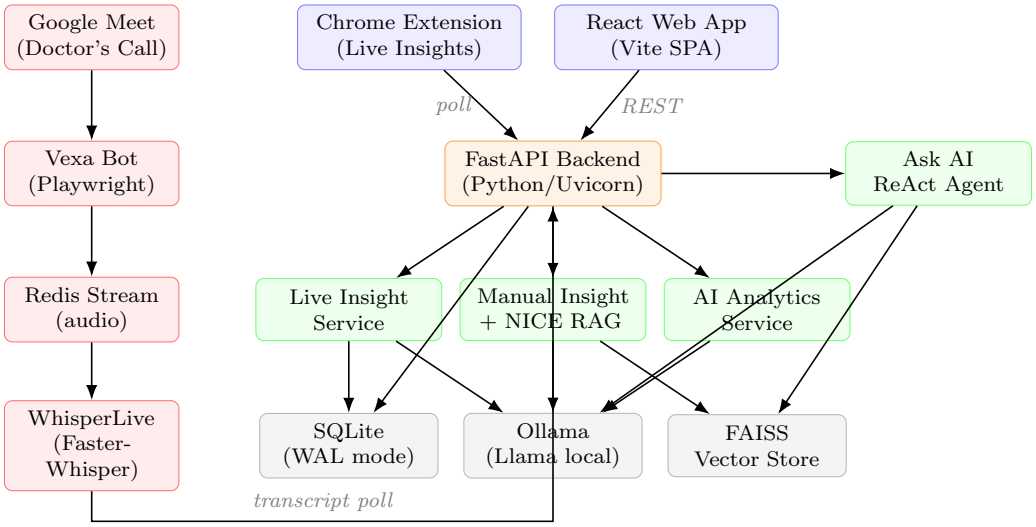


Figure 1: System architecture of Med Assist. Red nodes run in Docker (Vexa); blue nodes are client-side; orange is the FastAPI backend; green nodes are AI service modules; grey nodes are data stores. All arrows represent localhost communication.

4.3 Technology Stack

Layer	Technology	Why
Backend API	FastAPI (Python)	Async I/O; `BackgroundTasks` keeps LLM inference off the request path
Database	SQLite + SQLAlchemy (WAL)	Zero setup; WAL mode allows concurrent writes from background workers
Frontend	React + Vite	SPA model with fast hot-reload for iterative development
Chrome Extension	Manifest V3 side panel	Native tab-level integration; shares the web app's session cookie
Meeting bot	Vexa.ai (Docker Compose)	Open-source, self-hosted, works with Google Meet
Transcription	Faster-Whisper (`small` model)	CPU-feasible streaming ASR with VAD and stable-hypothesis gating
Audio bus	Redis Streams	Decouples audio capture from transcription without backpressure
LLM (insights)	Ollama + Llama 3.2 (3B)	Fits in 16 GB RAM; completes a live inference call in under 15 s on CPU
LLM (Ask AI)	Ollama + Llama 3.1 (8B)	8B scale is needed for reliable ReAct tool-calling
Embeddings	`all-MiniLM-L6-v2`	Runs on CPU with no external calls; fast semantic similarity
Vector store	FAISS (in-memory / persisted)	No external service; per-patient cache avoids repeated index builds
Agent framework	LangChain + LangGraph	ReAct reasoning loop with typed tool definitions
NICE RAG index	FAISS (persisted)	Pre-built index loaded from `data/vectorstore/guidelines`

Table 1: Technology choices and the reasoning behind them.

5 Implementation of Core Features

5.1 Meeting Bot, Audio Capture, and Transcription

5.1.1 Phase 1: Chrome Extension

The prototype used a Chrome MV3 side panel with `chrome.tabCapture.getMediaStreamId` for audio capture. Two alternatives were tried and rejected: `getDisplayMedia` produces a `DOMException` under MV3 restrictions in a side panel, and `chooseDesktopMedia` requires a manual tab-selection popup on every recording. Audio was encoded as WebM blobs and posted via multipart HTTP to `POST /consultations/{id}/audio`. The backend saved chunks to a `NamedTemporaryFile` before sending them to the OpenAI `whisper-1` API. The pipeline worked, but sending audio to OpenAI's servers was the fatal flaw.

5.1.2 Phase 2: Recall.ai

The backend posted the Google Meet URL to Recall.ai's `/bot` endpoint and received transcripts via webhook as the agent joined the call. A resilient search pattern handled the inconsistent response schema from the webhook response. There was an async bug with `leave_call()` which blocked the FastAPI event loop while waiting for Recall.ai to acknowledge the bot had left. It was fixed by making it properly async. Recall.ai's cloud still handled all the audio, so it had to go.

5.1.3 Phase 3: Self-Hosted Vexa.ai

Vexa.ai runs as a Docker Compose stack: an **api-gateway** on port 8056, a **bot-manager**, a **vexa-bot** container running Playwright-driven Chromium, a **whisperlive-cpu** container on port 9090 running Faster-Whisper (`small` model in CPU mode), a **Redis** message bus, a **transcription-collector** on port 8123, and an internal **Postgres** instance for Vexa's own metadata.

When a consultation starts, the backend extracts the native Google Meet ID via regex and posts to `localhost:8056/bots` with `voice_agent_enabled: false` — listen-only, no outbound audio. The meeting ID goes into `consultations.recall_bot_id`. A sync service polls `localhost:8123/transcripts/google_meet/{id}`, matches speakers to Doctor or Patient by comparing emails to the authenticated user's account, and writes `TranscriptEntry` rows. When the consultation ends, a `DELETE localhost:8056/bots/google_meet/{id}` removes the bot.

Inside Vexa, the Playwright browser joins the meeting as a named participant, intercepts audio via a virtual sink, and publishes raw chunks to Redis. The `whisperlive-cpu` container reads from Redis, applies Voice Activity Detection (VAD) with `MIN_AUDIO_S=1.0 s` to filter silence, runs Faster-Whisper, and holds back results until a `SAME_OUTPUT_THRESHOLD` gate confirms the hypothesis has stabilised.

The trickiest engineering problem was **temporal synchronisation**. Faster-Whisper keeps revising earlier hypotheses as more audio arrives. A naive "overwrite everything on each poll" approach corrupted transcripts by deleting confirmed text. The fix: calculate the oldest timestamp in the incoming payload (`min_start`), delete only the entries that fall within that window, and write the new refined versions. Earlier confirmed entries are left alone.

5.2 AI Insights: Manual and Live Modes

Table 2 shows the main differences between the two pipelines.

Attribute	Manual (Post-Consultation)	Live (Real-Time)
Trigger	Doctor clicks Generate after the call ends	Background task fires automatically every 12 s
Data source	Full transcript (up to 8000 chars) + EHR + NICE RAG (k=3)	New transcript delta + established facts list + 800-char bridge
NICE RAG	Yes, via FAISS	Not included (too slow on CPU)
`num_ctx`	6000	2048
`num_predict`	700	400
Temperature	0.0	0.0
Storage	`ai_insights` (one per consultation)	`live_insights` (one per worker run)
Output	summary, key_points, suggested_questions, risk_flags	summary, new_problems, suggestions, risk/-context

Table 2: Manual vs. Live insight pipeline parameters.

5.2.1 Manual AI Insights

The pipeline collects three things: the patient's name, date of birth, gender, and `key_history`; all `TranscriptEntry` rows sorted by timestamp and formatted as `[speaker]: text`, capped at 8000 characters from the tail to prioritise recent dialogue; and the top-3 NICE guideline chunks retrieved from the persisted FAISS index using the consultation's `about` field plus the first 600 transcript characters as the query. The LLM configuration is `temperature=0`, `keep_alive=-1` (the model stays in RAM permanently, avoiding cold-start delays), `num_ctx=6000`, `num_predict=700`, and `repeat_penalty=1.2`.

The scope of the NICE retrieval query produced a few problems. Early versions searched NICE guidelines using the patient's entire medical history, which caused wrong results—like suggesting knee arthritis guidelines for a patient with a urinary tract infection. I fixed this by limiting searches to just the reason for the visit (the 'about' field) and first few lines of the consultation

Two prompt decisions had the biggest impact on quality. First, the strict grounding rule: "Never infer, imply, or assume medical diagnoses. If a condition is not explicitly spoken, it does not exist." This stopped the model from hallucinating comorbidities. Second, the output format constraint requiring a plain JSON object with no markdown fencing made parsing deterministic. Earlier prompts that included example symptoms caused the model to copy those examples verbatim into unrelated consultations — removing all examples fixed that entirely.

5.2.2 Live AI Insights

The live pipeline uses differential processing (sending only new conversations) to keep prompt size constant as the consultation grows. Each background task run takes only the transcript entries created after the last `LiveInsight` timestamp (the delta). All previous `LiveInsight` records are flattened into a de-duplicated bullet list that tells the model what has already been reported. Only the last 800 transcript characters are included as bridging context. This design let `num_ctx` be cut from 4096 to 2048, halving memory pressure and speeding up inference.

Concurrency gets handled at two levels. A global `PROCESSING_CONSULTATIONS` set in memory prevents a second background worker from starting while one is already running for the same consultation. At the database level, the service commits and closes its SQLAlchemy session before the LLM call, releasing the WAL lock so Vexa's transcript sync can write in parallel. A fresh session opens after inference to save the result.

If all four output fields come back as empty strings, nothing is saved. This keeps the database clean during small talk. The throttle interval was reduced from 20 to 12 seconds during iterative

testing once the differential strategy brought per-call latency down enough to support the faster cadence.

5.2.3 Chrome Extension

The extension reads the `logged_in` cookie set by the React app — no separate login. It polls `GET /consultations/active` every 8 seconds. When an active consultation appears, it switches to live mode and begins polling `GET /insights/live/{id}` every 5 seconds. A local `Set` of rendered insight IDs prevents the same insight being displayed twice. `clearInsightsState()` runs on consultation end and on URL change, which fixed a race condition where ending one call and immediately joining another caused the previous patient's insights to appear in the new session.

Insights are displayed as expandable cards (Summary, Problems, Suggestions, Risks), prepended so the most recent is always at the top. The web app and extension share a *Midnight Medical* design system using CSS custom properties for dark/light mode theming, glassmorphism panels, and the Inter typeface.

5.3 AI Analytics

The weekly analytics module never reads raw transcripts. Its job is to read the `key_points` already produced by Manual Insights for each consultation in the week and summarise across them. The endpoint takes a `target_date`, calculates the ISO week Monday, queries all completed consultations in that window for the logged-in doctor, and concatenates the `key_points` into a LangChain prompt. The exact patient and consultation counts are injected as numeric variables. Temperature is 0.3 here — not 0.0 — because the output is narrative prose and a small amount of variation makes it more readable. The result is saved so future page loads do not need to call the LLM again.

Four bugs appeared during development and were fixed:

(1) **The plurality bug.** A single-patient week was described as "a large group of individuals presented with shoulder pain." Fix: inject the exact count and add the rule "if consultation_count is 1, do not use plural phrases."

(2) **Background history reported as acute complaints.** A patient attending for a UTI had their history of migraines listed as a new acute problem. Fix: an explicit prompt rule separating acute complaints from background risk factors.

(3) **Healthy-patient collapse.** When a patient had no problems, the model produced malformed JSON trying to classify demographic details as clinical risks. Fix: an escape hatch in the Manual Insights prompt — "if the patient is healthy, output: Routine Wellness Exam: No acute problems."

(4) **NICE RAG misfires** on background conditions, fixed as described above.

5.4 Ask AI

5.4.1 Architecture

Ask AI uses a LangGraph `create_react_agent` [10] with one tool: `get_patient_records`. The agent decides whether to call the tool, invokes it with the patient name and a query string, gets back structured context, and writes an answer. The tool fuzzy-matches the patient name against both `name` and `display_id` with a case-insensitive `ilike` query (so "John" matches "John Smith"), fetches all consultation and transcript records, chunks transcripts into 600-character segments with 100-character overlap using `RecursiveCharacterTextSplitter`, embeds them

with `all-MiniLM-L6-v2`, and runs a top-5 FAISS similarity search.

The system deliberately queries raw transcripts rather than the `AIInsight` summaries. If the agent relied on another model's summaries, any hallucination in that earlier stage would compound here. Raw transcripts are the ground truth.

Clinical guardrails tell the agent to refuse diagnosis or prescribing requests, say "This is not documented in the available notes" for missing information, and handle unknown patients gracefully. Importantly, the prompt also tells the model not to refuse valid clinical questions on "medical advice" grounds — that over-cautious refusal pattern is common in safety-trained models and is unhelpful when the doctor is asking about their own patient's factual history.

5.4.2 Model Selection

Llama 3.2 (3B) scored 0.18 on faithfulness in RAGAS evaluation. The problem was tool-calling: instead of sending natural-language queries to the FAISS tool, the model sent raw JSON dictionaries like {`"date_of_birth": "date"`}. FAISS cannot parse JSON, so retrieval returned nothing, and the model hallucinated the answers. Llama 3.1 (8B), which Meta trained specifically for tool-calling, sent clean natural-language queries like `query="demographic background"` and retrieved the right content. Faithfulness went from 0.18 to 0.75.

Metric	Llama 3.2 (3B)	Llama 3.1 (8B)	Notes
Faithfulness	0.18	0.75	+317%; answers grounded in retrieved data
Answer Relevancy	0.39	0.68	+74%; responses address the actual question
Context Precision	0.09	0.39	+333%; correct transcript chunks retrieved

Table 3: RAGAS results for Ask AI (GPT-4o as judge), tested against 4 patients, 20 consultations, and 3 adversarial questions.

5.4.3 Engineering Optimisations

Four things were needed to make the 8B model run acceptably on CPU. **Context truncation**: the endpoint only injects the last 6 chat messages. **FAISS per-patient cache**: `patient_vectorstore_cache` is keyed by patient ID so the expensive embedding step only happens on first query; follow-ups are instant. **LLM tuning**: `temperature=0` (deterministic), `num_predict=200` (short answers — this is a lookup tool), `num_ctx=4096`, `num_thread=8` (matches the Core Ultra 7's performance core count), and `keep_alive=-1` so Ollama never unloads the model between requests. **Auto-titling**: the first 30 characters of the opening message become the session title automatically.

6 Evaluation

Every AI feature was graded quantitatively before and after each change.

6.1 Manual Insights: Transcript Comparison

Method. An automated script (`test_ai_analytics_evaluation.py`) simulated 1 to 12 patients per week. Each generated insight was checked against the source transcript for dropped facts, invented symptoms, and formatting problems.

What went wrong early. The model hallucinated comorbidities (assuming a diabetic patient also had hypertension with no transcript evidence) and copied illustrative examples from the system prompt into unrelated consultations. Both issues disappeared after removing all examples

from the prompt and adding the anti-inference rule. The RAG misfire on background conditions was fixed by scoping the retrieval query to the complaint. JSON parsing errors were fixed by adding "Return ONLY a valid JSON object. No markdown, no explanation."

Final score: 7.5/10. One edge case remains: when a patient attended specifically to discuss adjusting their medication dosage, the model mentioned the drug despite a "Do NOT mention medications" formatting rule. That is arguably the right call — omitting the medication would have made the summary factually incomplete. The model prioritised accuracy over formatting, which is a safe failure.

6.2 Live Insights: GPT-4o Critic Agent

Method. A custom test suite (`test_live_insights_eval.py`) ran three scenarios through the live pipeline and sent each 12-second delta to a GPT-4o critic for grading (0–10 per sub-dimension):

- **Scenario A — Cardiology.** Chest pain with a mid-consultation correction (stomach pain revised to chest pain) and a late disclosure (daily aspirin mentioned near the end).
- **Scenario B — Dermatology.** Suspicious mole with a family history of melanoma stated in a single 116-character sentence.
- **Scenario C — Psychiatry.** Generalised anxiety including an unrelated complaint about a broken-down car. The car should produce no clinical output.

Initial scores were below 3/10 across all dimensions. Four fixes brought the average to 7.5/10:

Fix 1 — Removing the 120-character minimum (biggest single gain). The original code skipped any transcript segment shorter than 120 characters to save CPU time. The 116-character melanoma sentence in Scenario B was silently dropped. Risk Tracking scored 0/10 for that run. Removing the threshold and relying only on the 12-second time-throttle fixed it immediately: the model caught the melanoma family history on the next cycle and scored 10/10.

Fix 2 — Prompt consolidation and JSON enforcement. The early prompt gave the 3B model too much structural freedom. It returned nested JSON, added markdown fences that broke the parser, and mixed new facts with already-reported ones. A hard constraint — "Respond ONLY with a valid JSON object. No markdown, no explanation" — and a code-level sanitiser that strips fences before parsing resolved all of that.

Fix 3 — Differential context. Passing the full growing transcript every cycle caused tokens to grow without any bounds. The model lost track of what it had already reported and started repeating itself. Sending only the new transcript every 12 seconds with an established-facts list, plus 800 chars of bridge context keeps prompt size constant regardless of consultation length and let `num_ctx` be cut from 4096 to 2048.

Fix 4 — Silence preference and empty-output guard. Without instruction the model generated output during pure small talk. In Scenario C it classified the car breakdown as a clinical problem. Adding a silence-preference rule to the prompt and skipping the database write when all fields are empty cleared most of that.

Metric	Score / 10	Notes
Summary Accuracy	7.78	Strong narrative continuity across deltas
Problem Extraction	6.78	Some residual small-talk leakage; weakest dimension
Suggested Questions	8.33	Highest score; clinically plausible and well-timed
Risk & Context Tracking	7.22	Rose from 0/10 to 7.22 after removing the character filter
Average	**7.53**	

Table 4: Live Insights evaluation scores from the GPT-4o critic agent.

Two problems remain that cannot be fixed by prompt engineering. The **Eager Summariser**: the 3B model occasionally documents small talk as clinical output despite the silence rule — this is a known limitation of small models with negative constraints. The **Ghost Memory**: when the patient corrects "stomach pain" to "chest pain", the model adds the new symptom correctly but does not reliably delete the original. Both are properties of 3B-scale models. Swapping to a frontier LLM such as Claude 3.5 Sonnet or GPT-4o would, according to the evaluation reports, push scores above 9.5/10 without changing any of the architecture.

6.3 Weekly Analytics: Comparative Check

Method. Summaries for 1 to 12 simulated patients were checked against the source Manual Insights for invented trends, wrong plurals, and inaccurate caseload counts.

Final score: 8.0/10. This is the highest score of the three modules. Working from pre-structured `key_points` rather than raw dialogue makes hallucination harder — the model can only reorganise what is already there. Deductions come from the same Negative Prompt Problem seen elsewhere: the model occasionally breaks a formatting rule when following it would mean omitting something clinically obvious.

6.4 Ask AI: RAGAS

Method. The RAGAS framework [11] used GPT-4o as a judge. The test pulled 4 real patients and 20 consultations directly from the SQLite database, plus 3 adversarial trick questions (a chocolate cake recipe, investment advice, medication dosing). RAGAS measures Faithfulness (are answers grounded in what was retrieved?), Answer Relevancy (does the response address the question?), and Context Precision (did retrieval return the right chunks?).

Results are in Table 3. The 317% faithfulness jump is the clearest quantitative result in the project. It shows that the RAG architecture is sound but worthless if the agent model cannot formulate tool queries — and that 3B models cannot. Context Precision (0.39) is the remaining bottleneck, caused by `all-MiniLM-L6-v2`'s limited semantic understanding on clinical vocabulary. All three adversarial questions were correctly refused.

6.5 Reflection

What worked. The evaluation methodology — automated scripts, an independent GPT-4o critic, and RAGAS — produces numbers that can be traced back to specific code changes, which is more reliable than informal testing. The Vexa migration addressed a real regulatory constraint rather than a theoretical one. The "summary of summaries" design for Analytics keeps both context-window size and hallucination risk manageable. The differential strategy for live insights is a clean solution to the prompt-growth problem.

What did not. Small models fail at negative constraints consistently across all three AI features. Context Precision in Ask AI (0.39) shows that the embedding model is the bottleneck, not the

agent logic. The system has not been near a real clinical setting: all evaluation used scripted mock transcripts.

7 Limitations and Future Work

7.1 Current Limitations

No NICE retrieval in Live Insights. The 12-second CPU budget makes a FAISS retrieval step impractical during the live pipeline. NICE-informed suggestions are only available post-consultation.

Hardware ceiling. Running Vexa's WhisperLive, FastAPI with background workers, and Ollama simultaneously on one laptop with no GPU constrains everything. Models larger than 8B took 60–120 seconds per inference call or ran out of memory entirely during testing.

Negative-constraint failures. The Eager Summariser and Ghost Memory are properties of 3B-scale models, not of the pipeline design. The architecture does not need to change to fix them.

Synthetic data. No real clinicians were involved and no real PHI was processed. Whether the insights would be useful to an actual GP is unknown.

Google Meet only. Vexa's bot is configured for one platform in the project.

7.2 Future Work

1. **Upgrade the LLM backbone.** A better model (GPT-4o, Claude 3.5 Sonnet, or a locally-hosted Llama 3 70B with sufficient GPU) would resolve the negative-constraint failures.
2. **Guideline hints in Live Insights.** Pre-loading a small speciality-specific NICE subset into the live system prompt — rather than a per-call FAISS retrieval — could deliver guideline awareness with a more powerful model that runs fast.
3. **Better embeddings.** Replacing `all-MiniLM-L6-v2` with a clinical domain model such as BioLORD [12] would target the Context Precision gap and is projected to push retrieval precision above 0.85.
4. **Clinician validation.** The system needs to be tested by actual GP registrars before any claims about clinical usefulness can be made.
5. **Multi-platform support.** Extending Vexa to Microsoft Teams and Zoom.

8 Conclusion

The project set out to build an AI consultation assistant that keeps all patient data local. It achieved that. The system went through three audio-capture approaches before settling on a fully self-hosted Vexa.ai stack, and it now runs Faster-Whisper transcription, FAISS retrieval, and Llama inference entirely on the clinician's own laptop with no PHI touching an external server.

The five modules cover the consultation lifecycle: Vexa handles transcription, Manual Insights covers the post-call documentation, Live Insights and the Chrome extension give in-meeting support, AI Analytics reviews the week's caseload, and Ask AI handles patient queries. Evaluation across all four AI features produced quantitative evidence of what works (7.5–8.0/10 overall, 317% faithfulness improvement after the model upgrade in Ask AI) and what does not (3B models and negative constraints are a bad combination).

The architectural lesson that I learnt was that model scale is a hard ceiling, not a prompt engineering problem. A 3B model is sufficient for structured summarisation under strict rules;

it is not sufficient for multi-step tool-calling or reliable negative-constraint adherence. Keeping each module narrowly scoped so that the model is only asked what it can reliably do — and reading from the cleanest available data source — turned out to matter more than any individual prompt change.

The next steps are straightforward: a more capable LLM backbone, a clinician validation study, and support for additional meeting platforms.

References

[1] NHS Digital (2023). *GP Patient Survey 2023*. NHS England. Available at: https://www.gp-patient.co.uk/

[2] Reeves, M.J., et al. (2008). "Divided attention in physicians: implications for electronic health record use and patient safety." *Journal of the American Medical Informatics Association*, 15(5), 593–599.

[3] Vexa.ai (2024). *Open-source meeting intelligence platform*. GitHub. Available at: https://github.com/Vexa-ai/vexa

[4] WhisperLive / Collabora (2023). *Real-time speech-to-text using Faster-Whisper*. GitHub. Available at: https://github.com/collabora/WhisperLive

[5] Britt, H., et al. (2022). "Patient data sovereignty in cloud-hosted clinical AI: risks and mitigations." *npj Digital Medicine*, 5, 78.

[6] Devaraj, A., et al. (2021). "Evaluation of automatic speech recognition and NLP for clinical documentation." *JMIR Medical Informatics*, 9(8), e27138.

[7] Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.

[8] Berner, E.S. (2007). *Clinical Decision Support Systems: Theory and Practice*. 2nd ed. Springer.

[9] National Institute for Health and Care Excellence (2024). *NICE Guidelines and Evidence*. Available at: https://www.nice.org.uk/guidance

[10] Yao, S., et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*.

[11] Es, S., et al. (2023). "RAGAS: Automated Evaluation of Retrieval Augmented Generation." *arXiv preprint arXiv:2309.15217*.

[12] Remy, F., Demuynck, K., & Demeester, T. (2022). "BioLORD: Learning Ontological Representations from Definitions for Biomedical Concepts and their Synonyms." *Findings of EMNLP*.

[13] Recall.ai (2026). *The API for Meeting Recording: Transcribe Zoom, Teams, and Google Meet*. Available at: https://www.recall.ai [Accessed March 2026].

[14] AssemblyAI (2026). *Speech-to-Text and Audio Intelligence API*. Available at: https://www.assemblyai.com [Accessed March 2026].